

EX. NO: 5**IMPLEMENTATION OF RSA ALGORITHM****AIM**

To implement the RSA encryption and decryption algorithm using Python.

DESCRIPTION

RSA (Rivest–Shamir–Adleman) is an asymmetric cryptographic algorithm that uses a pair of keys: a public key for encryption and a private key for decryption. It provides secure communication by ensuring that only the intended receiver can decrypt the message.

ALGORITHM

STEP-1: Choose two prime numbers **p** and **q**.

STEP-2: Calculate **n = p × q**.

STEP-3: Compute **φ(n) = (p - 1) × (q - 1)**.

STEP-4: Choose a public key **e** such that **1 < e < φ(n)** and **gcd(e, φ(n)) = 1**.

STEP-5: Compute the private key **d** such that **(d × e) mod φ(n) = 1**.

STEP-6: Encrypt the message using the public key.

STEP-7: Decrypt the ciphertext using the private key.

CODE

```
from math import gcd
```

```
# Function to find modular inverse
```

```
def mod_inverse(e, phi):
```

```
    for d in range(2, phi):
```

```
        if (d * e) % phi == 1:
```

```
            return d
```

```
# Input two prime numbers
```

```
p = int(input("Enter first prime number: "))
```

```
q = int(input("Enter second prime number: "))
```

```
n = p * q
phi = (p - 1) * (q - 1)

# Select public key
e = 2
while e < phi:
    if gcd(e, phi) == 1:
        break
    e += 1

# Generate private key
d = mod_inverse(e, phi)

print("\nPublic Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

# Message
msg = int(input("\nEnter message (number less than n): "))

# Encryption
cipher = pow(msg, e, n)
print("Encrypted Message:", cipher)

# Decryption
plain = pow(cipher, d, n)
print("Decrypted Message:", plain)
```

```
Output
Enter first prime number: 17
Enter second prime number: 11

Public Key (e, n): (3, 187)
Private Key (d, n): (107, 187)

Enter message (number less than n): 7
Encrypted Message: 156
Decrypted Message: 7

=== Code Execution Successful ===
```

RESULT

Thus, the RSA encryption and decryption algorithm was implemented successfully using Python.